

3.Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

CODE:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
struct Task {  
    int id;  
    int execution;  
    int period;  
    int remaining;  
    int deadline;  
};
```

```
void RMS(struct Task t[], int n, int timeLimit) {  
    printf("\nRate Monotonic Scheduling:\n");
```

```
    for (int time = 0; time < timeLimit; time++) {
```

```
        for (int i = 0; i < n; i++) {  
            if (time % t[i].period == 0)  
                t[i].remaining = t[i].execution;  
        }
```

```
        int selected = -1;
```

```
        for (int i = 0; i < n; i++) {  
            if (t[i].remaining > 0) {  
                if (selected == -1 ||  
                    t[i].period < t[selected].period)  
                    selected = i;
```

```

    }
}

if (selected != -1) {
    printf("Time %d : Task %d\n",
        time, t[selected].id);
    t[selected].remaining--;
}
else {
    printf("Time %d : Idle\n", time);
}
}
}

void EDF(struct Task t[], int n, int timeLimit) {
    printf("\nEarliest Deadline First Scheduling:\n");

    for (int time = 0; time < timeLimit; time++) {

        for (int i = 0; i < n; i++) {
            if (time % t[i].period == 0) {
                t[i].remaining = t[i].execution;
                t[i].deadline = time + t[i].period;
            }
        }

        int selected = -1;

        for (int i = 0; i < n; i++) {
            if (t[i].remaining > 0) {
                if (selected == -1 ||
                    t[i].deadline < t[selected].deadline)
                    selected = i;
            }
        }

        if (selected != -1) {

```

```

        printf("Time %d : Task %d\n",
               time, t[selected].id);
        t[selected].remaining--;
    }
    else {
        printf("Time %d : Idle\n", time);
    }
}
}

```

```

int main() {
    int n, timeLimit;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    struct Task t[MAX], temp[MAX];

    for (int i = 0; i < n; i++) {
        t[i].id = i + 1;

        printf("\nTask %d\n", i + 1);

        printf("Execution Time: ");
        scanf("%d", &t[i].execution);

        printf("Period: ");
        scanf("%d", &t[i].period);

        t[i].remaining = t[i].execution;
        t[i].deadline = t[i].period;

        temp[i] = t[i];
    }

    printf("\nEnter simulation time: ");
    scanf("%d", &timeLimit);
}

```

```
RMS(t, n, timeLimit);

for (int i = 0; i < n; i++)
    t[i] = temp[i];

EDF(t, n, timeLimit);

return 0;
}
```

Enter number of tasks: 3

Task 1

Execution Time: 1

Period: 4

Task 2

Execution Time: 2

Period: 5

Task 3

Execution Time: 1

Period: 8

Enter simulation time: 10

Rate Monotonic Scheduling:

Time 0 : Task 1

Time 1 : Task 2

Time 2 : Task 2

Time 3 : Task 3

Time 4 : Task 1

Time 5 : Task 2

Time 6 : Task 2

Time 7 : Idle

Time 8 : Task 1

Time 9 : Task 3

Earliest Deadline First Scheduling:

Time 0 : Task 1

Time 1 : Task 2

Time 2 : Task 2

Time 3 : Task 3

Time 4 : Task 1

Time 5 : Task 2

Time 6 : Task 2

Time 7 : Idle

Time 8 : Task 1

Time 9 : Task 3

Process returned 0 (0x0) execution time : 41.827 s

Press any key to continue.